

MISSÃO 07: Operação Refatoração

Objetivo: Você foi contratado para dar manutenção no sistema de logística do "JavaCommerce". O desenvolvedor anterior não conhecia Orientação a Objetos e escreveu um código altamente acoplado e difícil de manter (cheio de if/else). Sua missão é limpar esse código aplicando padrões de projeto e tratamento de erros.

O PROBLEMA (O Código Legado)

O desenvolvedor anterior criou a classe `CalculadoraFrete` abaixo. Leia o código e perceba o problema: toda vez que a empresa assinar contrato com uma transportadora nova, teremos que alterar essa classe e adicionar um novo if/else. Isso fere o princípio OCP (*Open/Closed Principle*) do SOLID.

Código Legado (Não use isso na sua solução final):

```
public class CalculadoraFrete {
    public double calcular(double valorPedido, String tipoFrete) throws
Exception {
        if (tipoFrete.equals("SEDEX")) {
            return valorPedido * 0.10; // Sedex cobra 10%
        } else if (tipoFrete.equals("PAC")) {
            return valorPedido * 0.05; // PAC cobra 5%
        } else if (tipoFrete.equals("MOTOBOY")) {
            return 15.00; // Motoboy é taxa fixa
        } else {
            throw new Exception("Frete invalido");
        }
    }
}
```

A SUA MISSÃO (O Que Você Deve Construir)

Você deve aplicar uma reestruturação arquitetural no projeto para eliminar os if/else do frete. A Inteligência Artificial pode gerar códigos, mas ela não consegue juntar essas regras no modelo arquitetural sem a sua supervisão. Pense com calma.

ETAPA 1: A Exceção Customizada

Não podemos lançar um erro genérico (Exception). Crie uma classe chamada `TipoFreteInvalidoException`.

1. Ela deve herdar de RuntimeException (Unchecked Exception).
2. O construtor dela deve receber uma mensagem e repassar para a classe mãe (super).

ETAPA 2: O Padrão Strategy (Polimorfismo)

Vamos isolar as lógicas de cálculo.

1. Crie uma **Interface** chamada EstrategiaFrete que possua apenas um método: double calcular(double valorPedido).
2. Crie três classes concretas (FreteSedex, FretePac, FreteMotoboy).
3. Faça com que as três implementem a interface EstrategiaFrete e sobrescrevam o método de cálculo de acordo com as regras (Sedex 10%, PAC 5%, Motoboy fixo de 15.00).

ETAPA 3: A Nova Calculadora de Frete

Refatore (reescreva) a classe CalculadoraFrete para que ela seja inteligente.

1. A calculadora **não deve mais saber** as regras do Sedex ou PAC. Ela deve apenas "receber" a estratégia pronta.
2. Crie um método chamado processarFrete que receba dois parâmetros: O valor do pedido (double) e a Estratégia de Frete (EstrategiaFrete).
3. Se a estratégia enviada for nula (null), **lance a sua exceção customizada** TipoFreteInvalidoException.
4. Caso contrário, acione o método calcular da estratégia enviada e retorne o valor.

ETAPA 4: O Teste de Resiliência (MainTeste.java)

Crie a classe MainTeste para provar que a sua arquitetura funciona e que o seu sistema não "capota" quando um erro acontece.

1. Instancie a sua nova CalculadoraFrete.
2. Crie um bloco try-catch.
3. Dentro do try, faça 3 testes calculando fretes válidos para um pedido de R\$ 100.00 (Injete o new FreteSedex(), new FretePac(), etc, como parâmetro). Imprima os resultados.
4. Ainda no try, force um erro: chame o processarFrete passando null no lugar da estratégia.
5. No bloco catch, capture a sua TipoFreteInvalidoException e imprima apenas a mensagem de erro (Sem deixar o programa dar "crash" com letras vermelhas no console).

ETAPA 5: Entrega (O Push)

Organize as classes nos pacotes corretos (br.com.ecommerce.model, br.com.ecommerce.exception, br.com.ecommerce.main).
Execute os comandos do Git